Engs 31 / CoSc 56 Digital Electronics Summer 2025
Final Project Report

Colin Wolfe and Elias Morley

Abstract

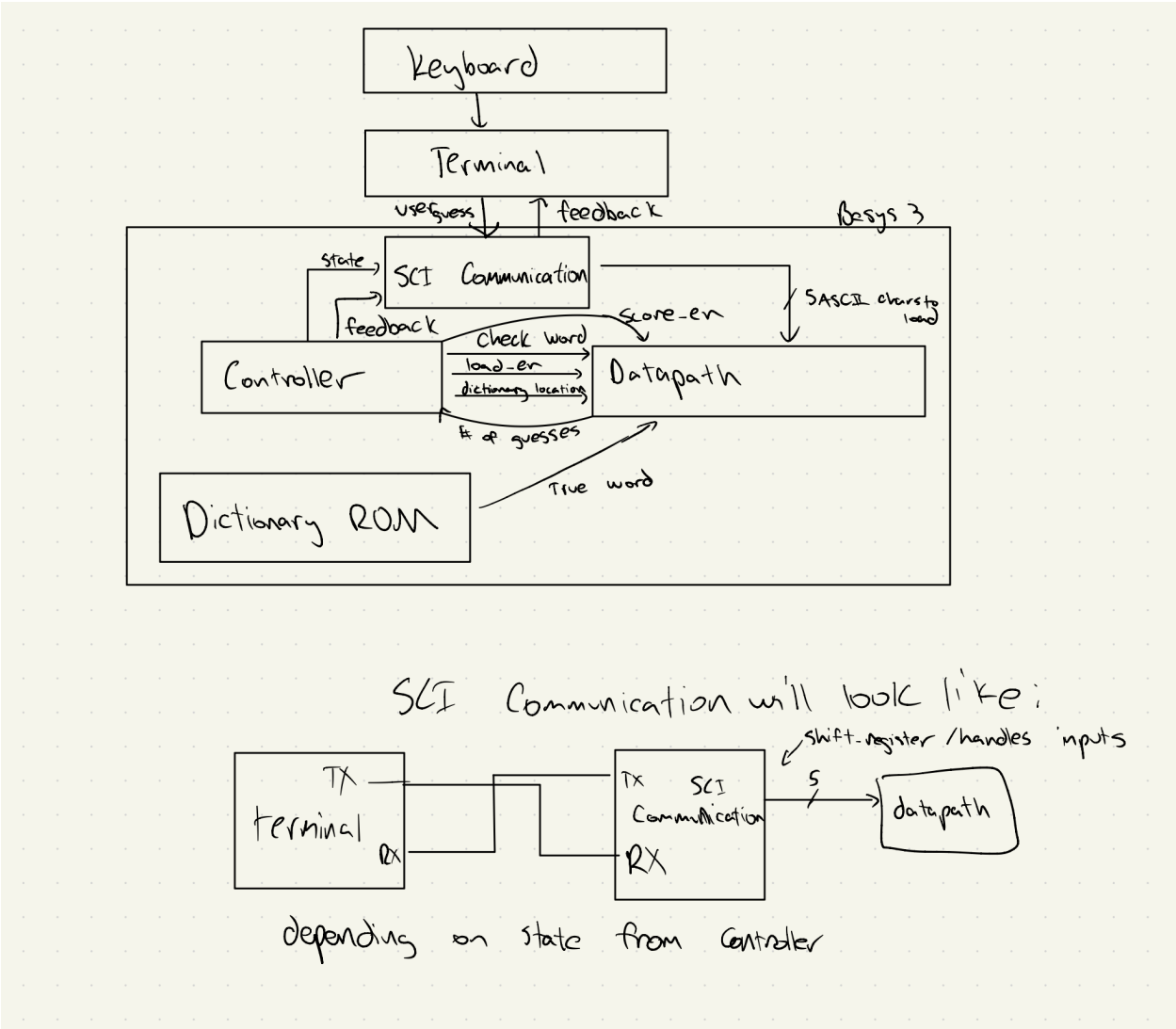
This project implements a digital hardware system that simulates the popular New York Times game Wordle on an FPGA. The design integrates serial communication, a datapath, and a finite state machine (FSM) controller to replicate the game's behavior. User input is entered via the terminal program PuTTY, transferred to the FPGA using SCI (serial communication interface), and processed within the system. The datapath executes the core comparison and grading logic, while the controller sequences the game through valid states. There is also a five character (each eight bits) buffer that temporarily stores the previous user guess. Most of these components were tested via testbenches, though the datapath grading was tested manually. We were successful in our undertaking, as we routed user input, correctly reverse engineered the Wordle algorithm, and then outputted it back to the terminal. There were some additions that could have been made to increase user experience and make the game more intuitive, but overall the necessary mechanics were there.

System Overview

The intended goal of the project is to successfully implement the popular game, Wordle, on an FPGA. The intended behavior of the system is straightforward: the user connects to the FPGA through PuTTY, enters a five-letter guess, and the system evaluates that guess against a hidden target word, just as the NYT does. The FPGA generates a grade consistent with Wordle's rules: letters in the correct position are marked !, letters present in the word but in the wrong position are marked ~, and letters not in the word are marked X. The user gets six guesses to guess the hidden target word.

The specs required to make the system successful are a system clock rate of 100MHz and a Baud rate of 9600 Hz for both PuTTy and the FPGA, allowing the device to operate in near real time.

Top-Level Block Diagram



Description of Ports

PORT Name	Direction	Function	Component
Clk	Input	System clock, std_logic;	Top Shell
Rx	Input	Receiving line for the serial communication interface, std_logic;	Top Shell

Tx	Output	Transmission line for the SCI, std_logic;	Top Shell

Description of Components

Top Shell:
The top level shell instantiates and connects the outputs of all of the other components. It defines the ports and proper routing of information throughout the system.

Datapath:
The datapath stores the current guess and does operations on it like checking it, seeing if that last guess won early and more. It can generate feedback and will pass the ! or ~ or X onto the transmitter.

Controller:
The controller is a finite state machine that outputs signals to control the datapath's operations. It cycles through various states of the game, like input guess, check guess, results, and then tells the datapath what to do. It is the brains of the input and grading process.

Buffer:
The buffer is a small 5×8-bit memory used to temporarily hold user guesses. It smooths the flow of data between the datapath and the SCI communication modules

SCI Transmitter:
The Serial communication interface transmitter sends out one bit at a time to the receiver. It uses the Baud rate, start bit, and stop bit, to get its message out.

SCI Receiver:
Receives in an 8-bit character, one at a time. Uses an FSM to determine whether or not to start the char, keep adding to the current char, stop the char, or keep waiting in idle.

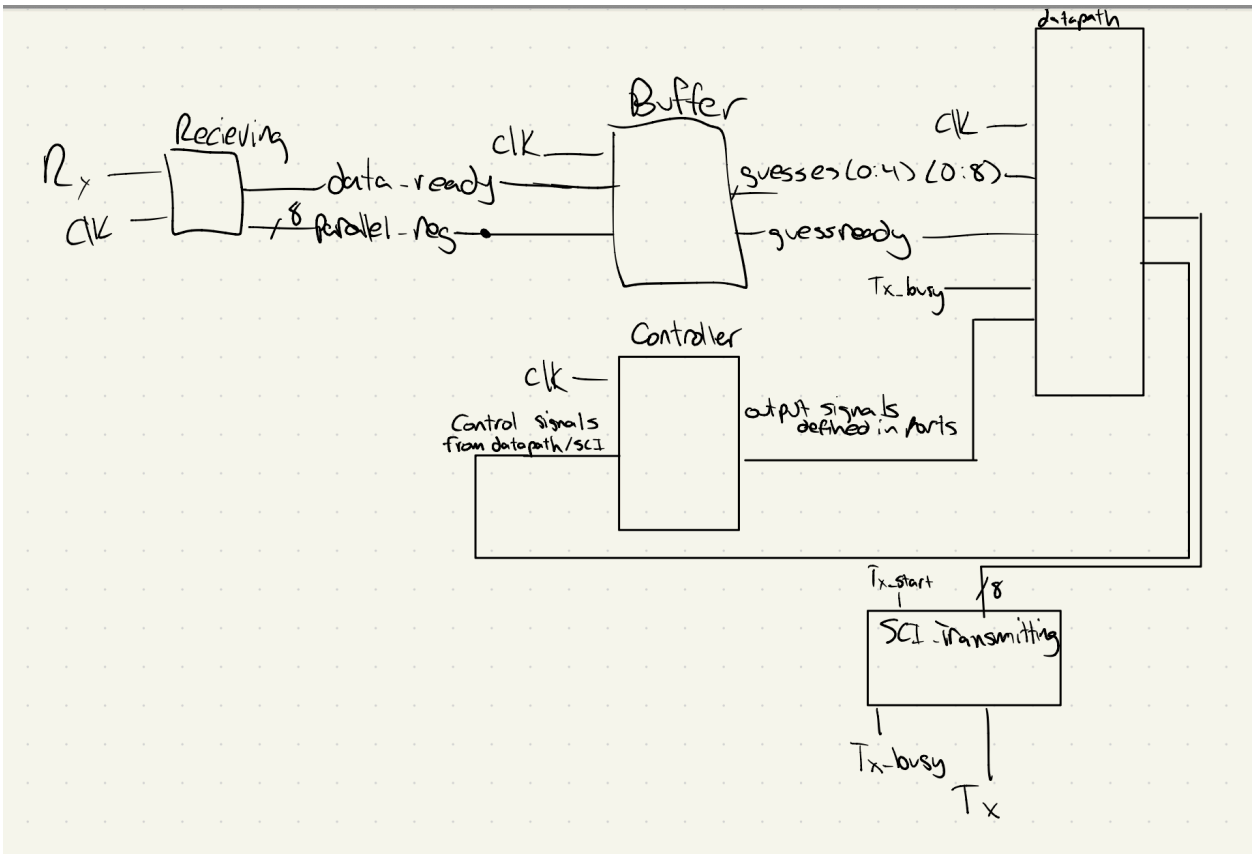
Technical Description

Top Shell:

The top level shell instantiates and connects the outputs of all of the other components. It is the interface between the real-world and the submodules.

Ports:

PORT Name	Direction	Function	Component
Clk	Input	System clock, std_logic;	Top Shell
Rx	Input	Receiving line for the serial communication interface, std_logic;	Top Shell
Tx	Output	Transmission line for the SCI, std_logic;	Top Shell



No finite state machine controlling the top level shell. Instead it mapped the instantiations together to link the proper inputs to one component with the outputs of another. This shell allows each component to handle the part it needs to, and interact with the user. There were a few special processes, like monopulsing some signals so as to not continuously guess or print or otherwise, but otherwise it was mostly mappings.

The only memory that was used was read-write memory, and was just used to store all input/output signals to and from the sub components. For instance, the buffer outputted a standard logic vector of size 8, so the shell needed to store a standard logic vector of size 8, so then it could route it to the datapath. That was all of the memory needed (just the exact inputs and outputs - same size and type - to the sub components).

Datapath:

The datapath stores the current guess and does operations on it like checking, generates feedback and will pass the ! or ~ or X onto the transmitter, and handles lots of the printing.

Ports:

Port name	Input/Output	Description (1 sentence + type)	Component
clk_port	Input	System clock driving all synchronous logic (std_logic).	datapath
load_guess_port	Input	Latches the five input guess bytes into internal registers (std_logic).	datapath
check_guess_port	Input	Triggers Wordle grading of the latched guess vs the hardcoded target (std_logic).	datapath
check_result_port	Input	Starts serialized printing of the graded result to TX (std_logic).	datapath
game_over_port	Input	Game over indicator (currently not used in processes here) (std_logic).	datapath
guess_port_1	Input	First character of the current guess (std_logic_vector(7 downto 0)).	datapath
guess_port_2	Input	Second character of the current guess (std_logic_vector(7 downto 0)).	datapath
guess_port_3	Input	Third character of the current guess (std_logic_vector(7 downto 0)).	datapath
guess_port_4	Input	Fourth character of the current guess (std_logic_vector(7 downto 0)).	datapath
guess_port_5	Input	Fifth character of the current guess (std_logic_vector(7 downto 0)).	datapath
take_tx_busy	Input	UART TX busy flag to pace byte launches (std_logic).	datapath
take_print_win	Input	Command to print the WIN message stream (std_logic).	datapath
take_print_lose	Input	Command to print the LOSE message stream (std_logic).	datapath
take_data_ready	Input	Pulse that a new RX byte is available for echo (std_logic).	datapath
take_parallel_reg	Input	The RX byte to echo if enabled (std_logic_vector(7 downto 0)).	datapath
take_echo_enable	Input	Enables echo of typed characters to terminal (std_logic).	datapath
take_reset_buffer	Input	External request to reset/clear buffer (declared; not used in this body) (std_logic).	datapath
print_to_terminal	Output	Byte presented to UART TX for transmission (std_logic_vector(7 downto 0)).	datapath
out_guess_check_done	Output	One-cycle strobe indicating grading finished (std_logic).	datapath
out_win	Output	High when all five letters are exact matches (std_logic).	datapath
out_done_printing	Output	One-cycle strobe when result line output is complete (std_logic).	datapath
out_tx_start	Output	One-cycle strobe to request TX of print_to_terminal (std_logic).	datapath
out_printed_win_result	Output	One-cycle strobe when WIN message stream completes (std_logic).	datapath
out_printed_lose_result	Output	One-cycle strobe when LOSE message stream completes (std_logic).	datapath
out_max_guesses_reached	Output	High when guess limit is reached (set in result path) (std_logic).	datapath
		*some generation from chat due to volume of descriptions	

Datapath RTL was extremely unwielding and is better thought of as pseudocode than an RTL. There are limited control signals coming from it as the component operates on the data and then simply sends out signals that it is doing so, without any real logic.

The datapath does not use a finite state machine, but instead relies on control signals from the controller such as load_guess_port, check_guess_port, and check_result_port to decide what operation to perform. When a new guess is loaded, the five character registers are updated, and on a check command the datapath compares those characters against the stored target word to generate match results. If the result command is given, the datapath sequences out the formatted feedback through a simple counter

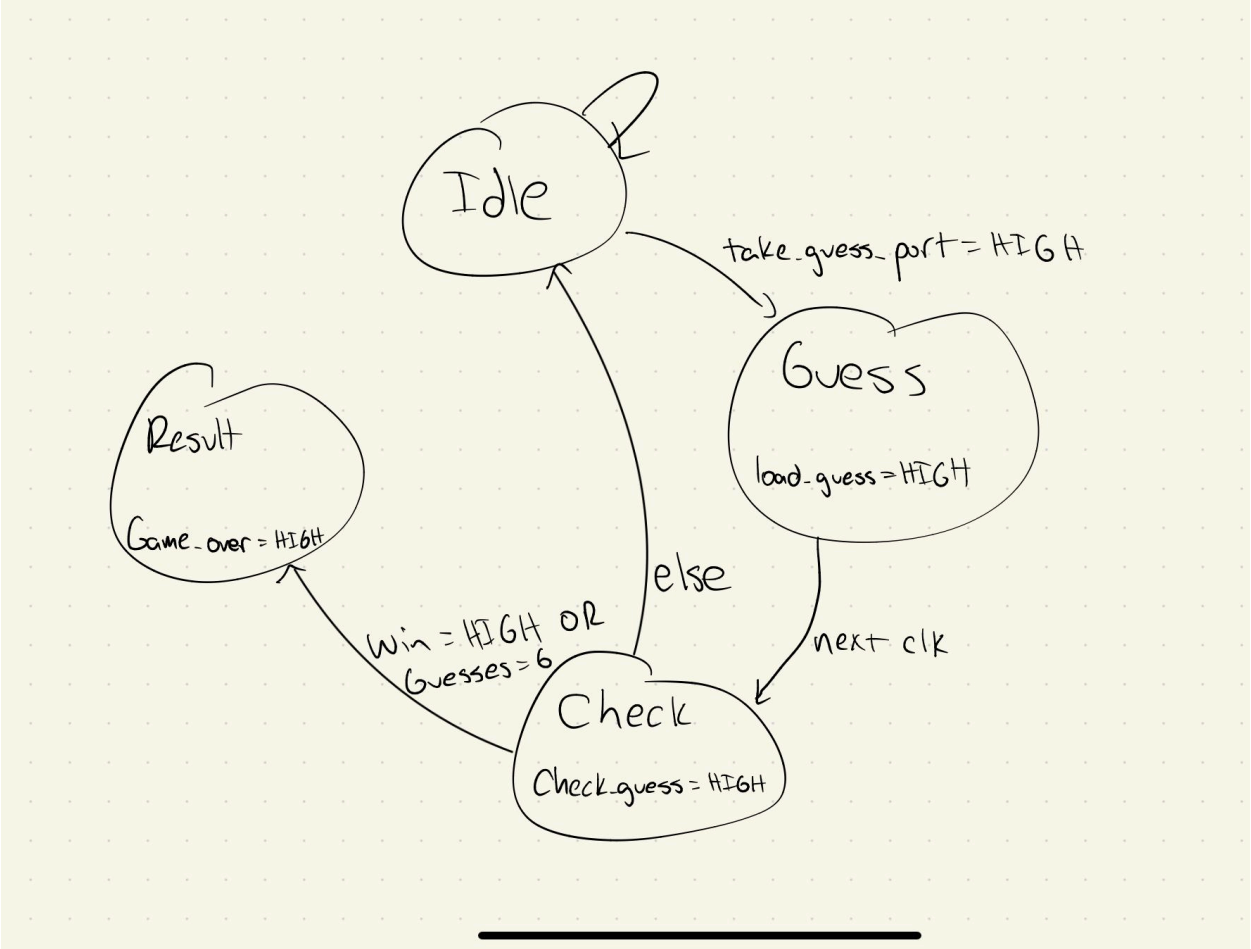
The datapath holds five eight-bit standard logic vectors for guesses, the hardcoded hidden word, and for the result vectors to be printed out. It also holds five two-bit standard logic vectors for seeing if a guess lines up exactly with the hidden letter, right letter and wrong spot, or wrong letter all together. There are also some other controls for logic, like a counter flag, a wait one cycle, a send active, and more.

Controller:

The controller is a finite state machine that outputs signals to control the datapath's operations. It cycles through various states of the game, like input guess, check guess, results, and then tells the datapath what to do.

PORT Name	Direction	Function	Component
Clk	Input	System clock, std_logic;	Controller
take_guess_port	Input	New guess submitted from buffer, std_logic	Controller
take_guess_check_done	Input	Guess checking is over, std_logic	Controller
take_win	Input	If current guess matches hidden word, std_logic	Controller
take_done_printing	Input	result printed to terminal, std_logic	Controller
take_print_win	Input	Win message printed,	Controller

		std_logic	
take_print_lose	Input	Lose message printed, std_logic	Controller
Load_guess, check_guess, game_over	Output	Control signals to the datapath based on the states, all std_logic	Controller
check_result	Output	Tells datapath to do the grading, std_logic	Controller
send_result_to_terminal	Output	Send the grades to the terminal, std_logic	Controller
Print_win, print_lose	Output	Either print that user won or lost, both std_logic	Controller
echo_enable	Output	Keystroke echo to PuTTy enable, std_logic	Controller
out_reset_buffer	Output	Clears input to prepare for new round, std_logic	Controller
take_max_guesses_reached	Output	No more guesses allowed, std_logic	Controller



It starts in IDLE, waiting for the buffer to signal that a guess is ready. From there it transitions to CHECK, where it directs the datapath to compare the guess against the hidden word. Depending on the outcome, it either loops back to accept the next guess or moves to END GAME if the player has won or used all six guesses.

The memory used in the controller is limited. It stores a type for current state and next state, so that it can properly progress the FSM. Also, it uses a number of guesses counter as a control signal for transition between check and either result or idle.

Buffer:

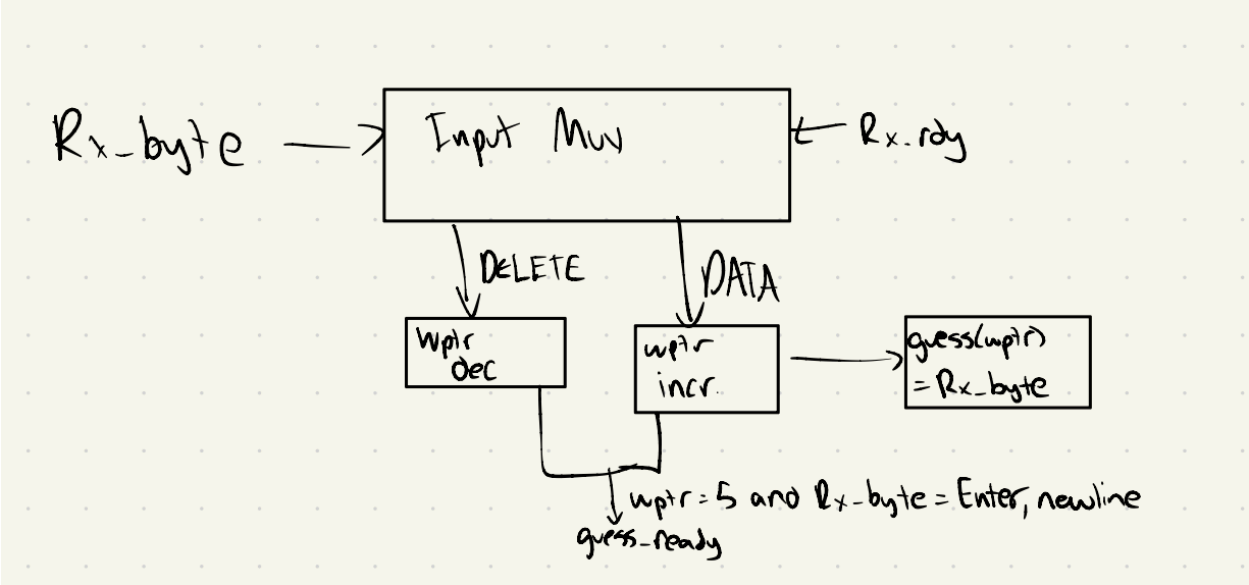
The buffer is a small 5×8-bit memory used to temporarily hold user guesses. It smooths the flow of data between the datapath and the SCI communication modules.

Ports:

PORT Name	Direction	Function	Component

Clk	Input	System clock, std_logic;	Buffer
Rst_buff	Input	Reset the buff, std_logic	Buffer
rx_byte	Input	Gets the byte from the SCI Receiver component, std_logic_vector 7 downto 0	Buffer
rx_rdy	Input	A signal that says when the byte from the Rx is ready, std_logic	Buffer
guess0	Output	The first character that the user guessed, std_logic_vector 7 downto 0	Buffer
guess1	Output	The second character that the user guessed, std_logic_vector 7 downto 0	Buffer
guess2	Output	The third character that the user guessed, std_logic_vector 7 downto 0	Buffer
guess3	Output	The fourth character that the user guessed, std_logic_vector 7 downto 0	Buffer
guess4	Output	The fifth character that the user guessed, std_logic_vector 7 downto 0	Buffer
wptr	Output	For debugging purposes, which char we are currently on, std_logic_vector 2 downto 0	Buffer

guess_ready	Output	Signal that says when all 5 chars are in, std_logic	Buffer



The Buffer does not use a finite state machine, instead it relies on the signals rst_buff and rx_rdy to determine when to take in another byte or to reset all of the current guesses. When a new guess comes in, if the guess is not the backspace, then it puts the Rx_byte into the next guess. If the Rx_byte is a backspace, then it moves the current index pointer back one, to delete the unwanted char. This allows for a user to update their guess as needed. Once 5 chars come in, then it sends out a guess ready signal, and stops accepting any new input until it's reset.

The buffer stores a two-dimensional array of dimensions five by eight, meaning five rows and eight columns. This corresponds to five chars and eight bits per char. Additionally it stores a 3 bit standard logic vector that represents the current index the guess is on. Thus we can store past guesses, make deletions, and know when to send out a guess ready signal. Both of these are read and write memory, as they are written to (or incremented in the wptr case) when the rx_rdy control signal comes in, and read from when wptr equals five. The control signal is the rx_rdy.

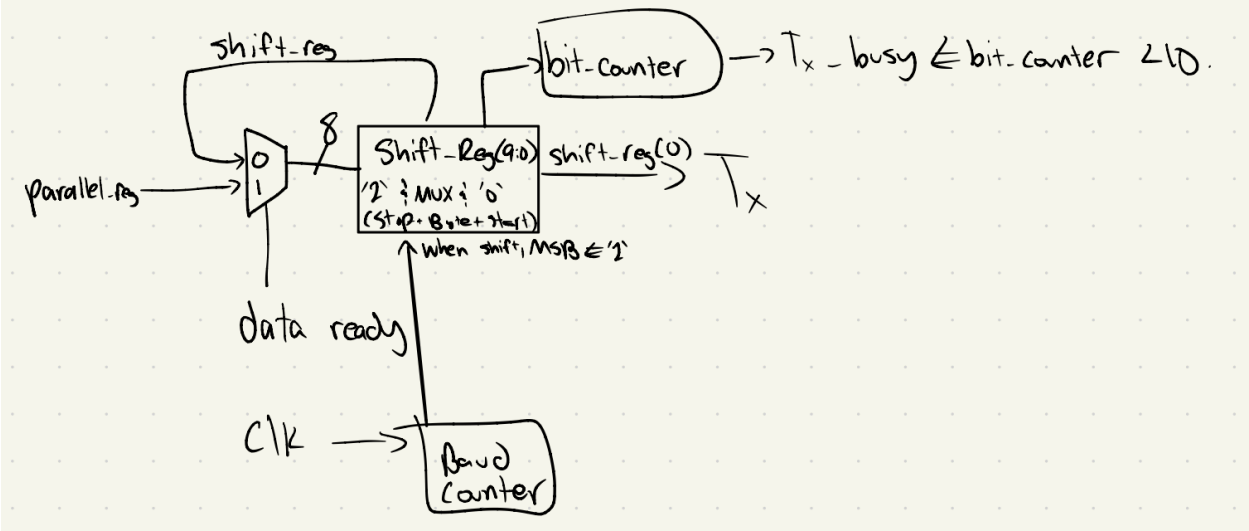
SCI Transmitter:

The Serial communication interface transmitter sends out one bit at a time to the receiver. It uses the Baud rate, start bit, and stop bit, to get its message out.

Ports:

PORT Name	Direction	Function	Component
Clk	Input	System clock, std_logic;	sci_transmitter
parallel_reg	Input	Receiving line for the serial communication interface, std_logic;	sci_transmitter
data_ready	Input	Tells the transmitter when to load in from parallel_reg and start, std_logic;	sci_transmitter
Tx_busy	Output	Alerts that the Tx is currently sending a message	sci_transmitter
Tx	Output	One bit output in the proper form for the uart sci with PuTTy, std_logic	sci_transmitter

RTL:



The transmitter does not have an FSM dictating its logic. The shift register will load in from parallel reg when the data_ready signal goes high, appending on a start and stop bit. Then it will shift out one bit at a time, incrementing the counter by one each time. Tx_busy stays high while the message is being sent out. After everything has been shifted out, you can set data_ready to be high again.

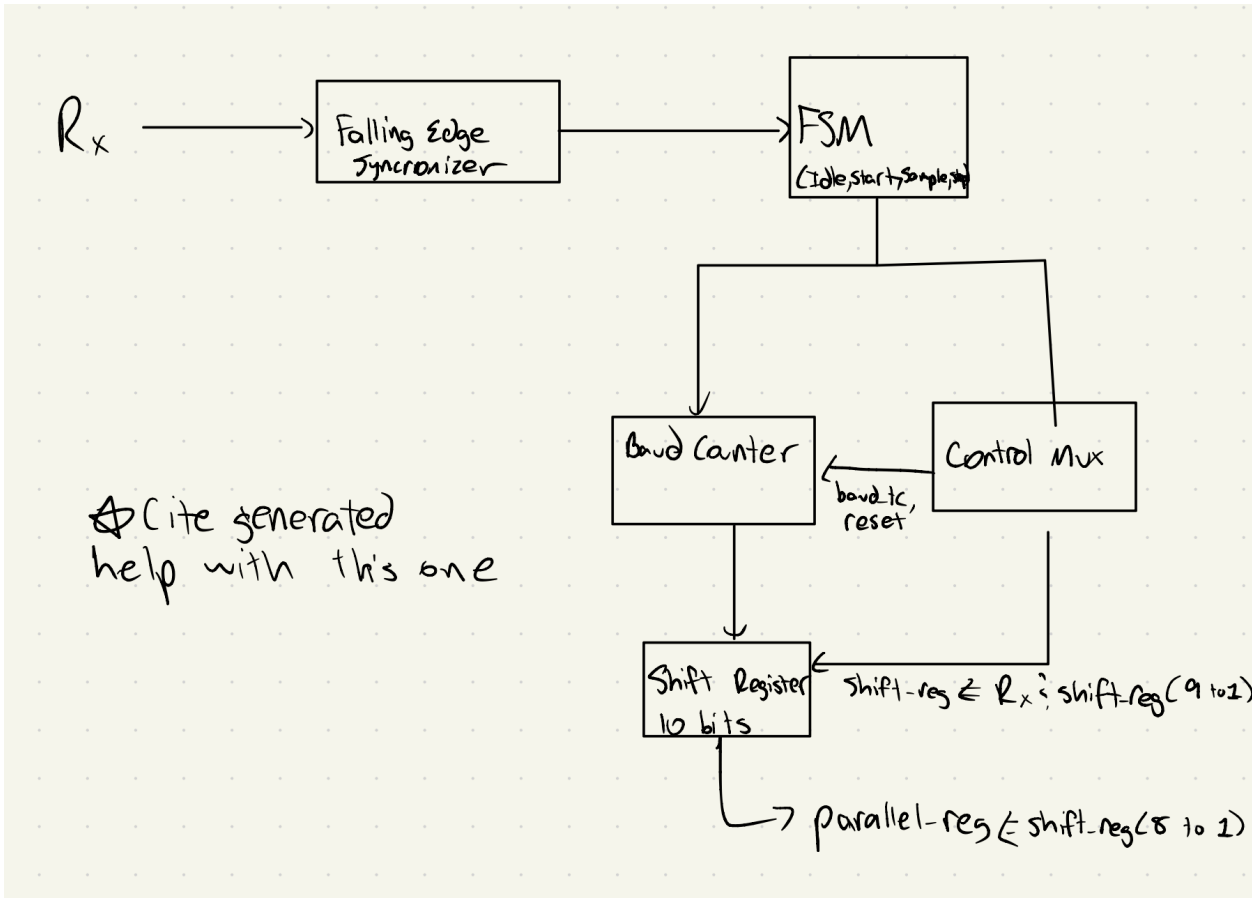
There is limited memory in use here. We are storing an integer for the Baud counter, a 10 bit standard logic vector for the shift register, and an integer for the bit counter to determine the Tx_busy.

SCI Receiver:

Receives in an 8-bit character, one at a time. Uses an FSM to determine whether or not to start the char, keep adding to the current char, stop the char, or keep waiting in idle.

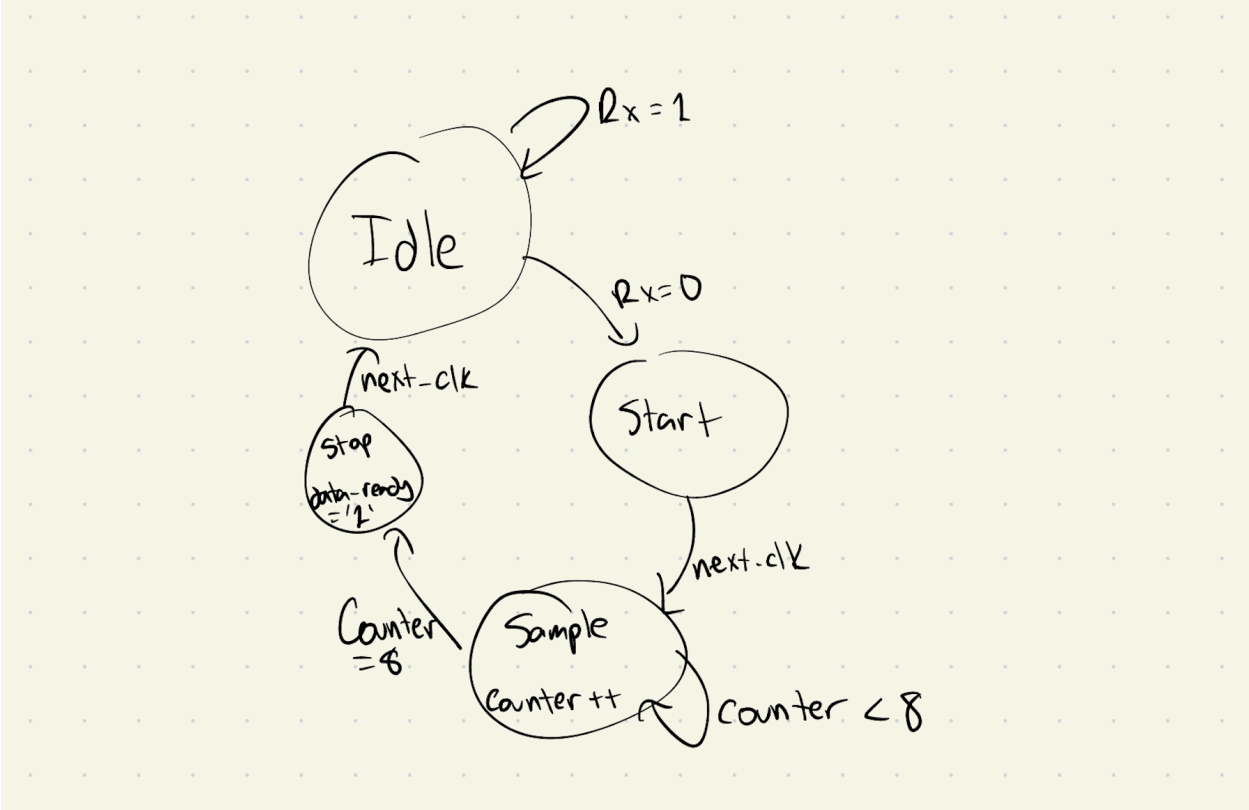
Ports:

PORT Name	Direction	Function	Component
Clk	Input	System clock, std_logic;	sci_receiver
Rx	Input	Receiving line for the serial communication interface, std_logic;	sci_receiver
data_ready	Output	Tells when the char has been fully loaded in, std_logic;	sci_receiver
parallel_reg	Output	The full 8 bit char to be used elsewhere (in the buffer), std_logic	sci_receiver



✡Cite generated help with this one

FSM:



The receiver uses a simple four-state FSM to control sampling of the serial input. In IDLE, it waits for a falling edge on Rx to detect a start bit. In START_BIT, it waits half a baud period and then samples to confirm. The FSM then enters SAMPLE, where it shifts in eight data bits every baud interval, before moving to STOP_BIT to sample the stop bit, assert data_ready, and return to IDLE.

The memory used is a 10 bit standard logic vector shift register to keep track of the serial input coming in, and then outputting the correct range (dropping the start and stop bit). Also it used a standard logic bit to keep track of the previous Rx, so that it can detect a falling edge. There were a few other control signals that were all standard logic bits related to moving properly through the FSM.

Testing Strategy

We built the project piece by piece, starting with SCI communication since we knew RX and TX had to work first. TX was the easy part because there was an example from class, and transmitting is simpler anyway. RX took a little more effort, but once both were built we tested

them with an echo setup (wiring RX directly into TX) and used PuTTY to make sure characters were coming through correctly.

After that we worked on the buffer. We needed a way to hold multiple characters, so we set up an array of vectors in the buffer file. We didn't bother with a testbench here either and just used PuTTY to check things. To tie everything together we made a simple SCI shell that managed RX, TX, and the buffer. Once we could take input from RX, pass it through the buffer, and send it back out TX, we moved on to the controller and datapath.

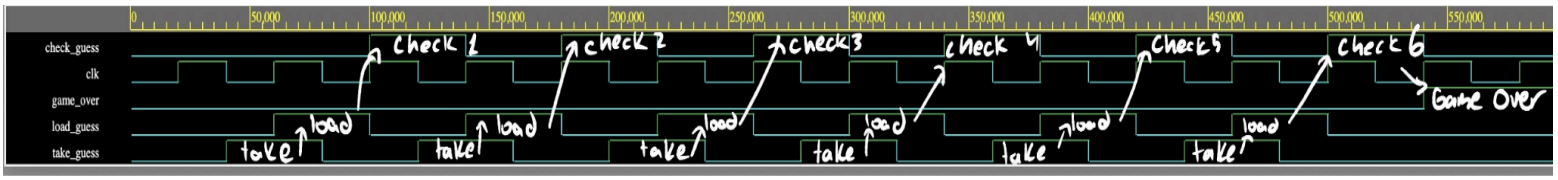
For the controller and datapath we leaned on our diagrams to map out how it should work. We started by testing with hardcoded guesses so we could make sure the Wordle logic functioned as expected. That part worked, we could print the right response for guesses, but again we relied only on PuTTY. When we tried to use real user input, things fell apart. The game would print results for each guess, but we couldn't actually detect win/lose states or print those messages. The shell had turned into a mess with too much functionality dumped into it, and the controller/datapath weren't strong enough to handle end states.

Eventually we decided a rewrite was the only way forward. We pulled the print logic out of the shell and into the datapath, gave the controller more states and signals, and finally wrote a real top-level testbench to properly test subcomponents. That cleaned up the structure a lot and gave us a more reliable system to work with.

Design Validation

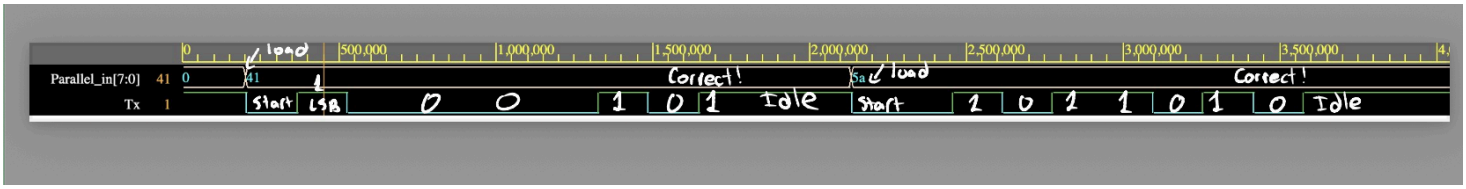
Controller

The testing strategy for the controller was to drive it with simulated inputs that mimic user guesses and then observe if it correctly asserted the load, check, and game over signals in the right order. This means the FSM needs to follow the intended sequence of states without skipping or hanging. A success looks like the signals toggling exactly as expected across multiple cycles. This test was a success!

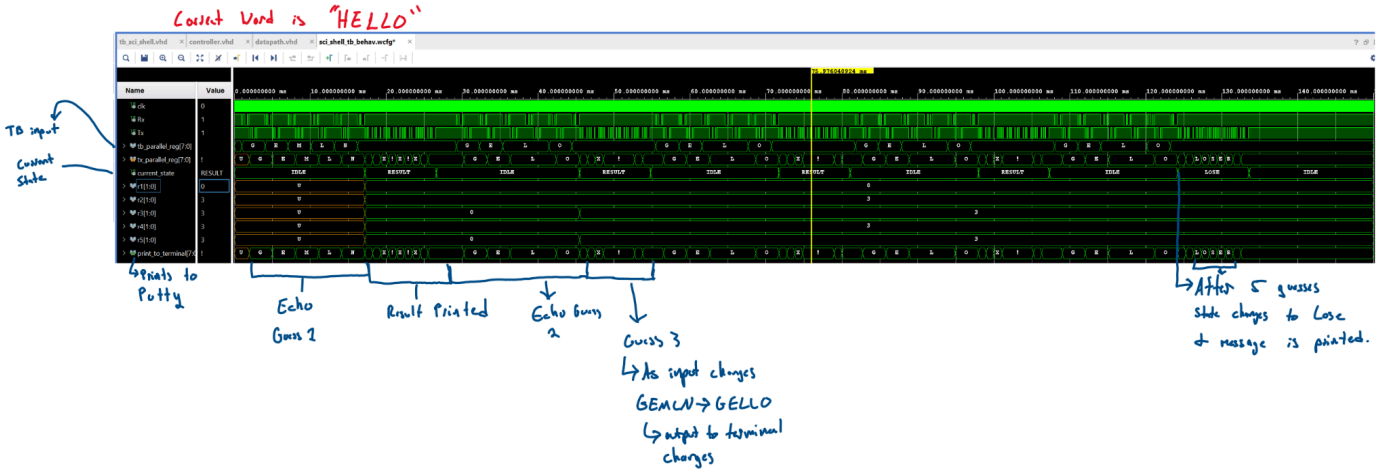


SCI Transmitter

The testing strategy for the transmitter was to give the test bench a parallel register and have it load it then, then to see if it properly outputted the correct Tx for it. This means standard UART protocol as well as a correct 8bit output. A success looks like the proper protocol is followed and the correct output is given. This test was a success!

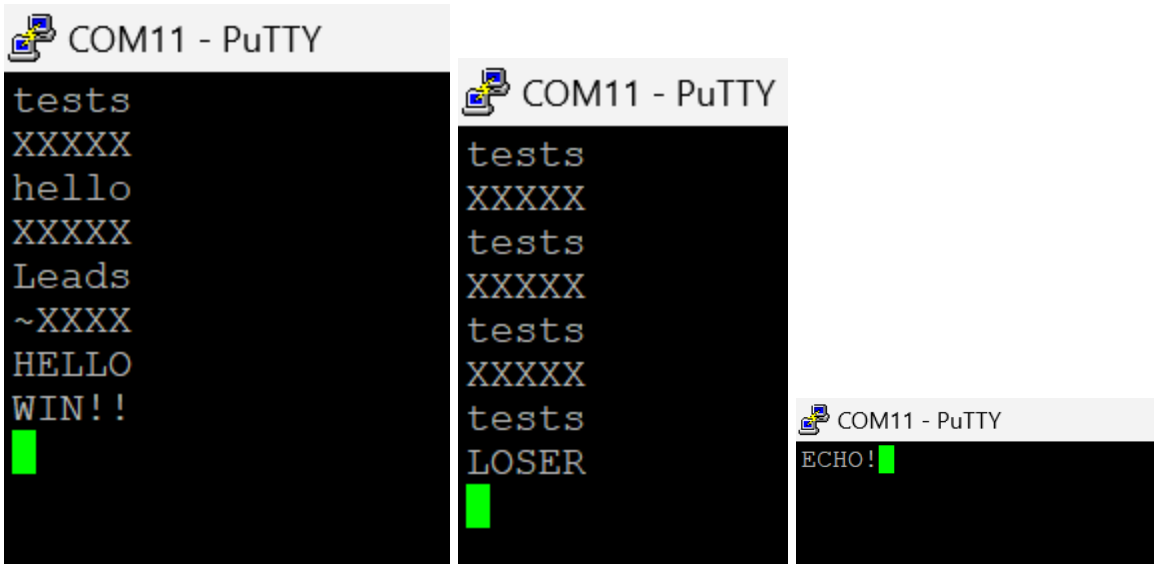


Top Level



Waveform that shows the state of the FSM progressing properly. Showing the letters being processed correctly. Shows the letters being sent out as well. In the end, at the bottom, you can see the lose state, correctly showing that LOSER is being outputted correctly, and that the game progressed as it should have.

Other tests showing full game ending in Win state; Showing full game in lose state; echoing to PuTTy



Behavioral

Barebones (older) demo of how it worked IMG_1375.MOV

Analysis of the Design

Resource Utilization

Our design made use of the FPGA's combinational logic (LUTs) to implement the controller's next-state logic, the datapath comparison checks, and the buffer control operations. It also relied on flip-flops to hold state, counters, and the five-character buffer used for storing guesses. In addition, the design utilized the FPGA's resources for SCI (UART) communication, which required shift registers and timing logic to serialize and deserialize characters between the FPGA and the terminal. We also used structural VHDL to modularize the design and combine components at the top level. Overall, the resource utilization was minimal compared to the FPGA's total capacity (no ROM, no signal processing, etc), leaving room for more complex features or scaling of the system.

4.2 Residual Warnings List any warnings that remain. Explain why they are safe to ignore. (Must resolve latch warnings!)
Some residual warnings

There are two latches, one for echo and the other for reset buffer. This latching is okay because, in the case of echo, we want to either always echo user input or never echo it, so latching onto this is fine (as it is just using a flip flop).

game_over signal is not used: we defined it during prototyping but forgot to delete it, easy one line fix (delete the signal declaration).
In the case of the reset buffer, we set it high when we need it to be reset, and low after, so it latches on to low while it waits to be set high again - which is perfectly acceptable.
A few other minor warnings related to types and signals not being used as we iterated through, which are also all acceptable (we converted types properly and sometimes changed signals throughout this process).

Division of Labor:

Colin designed and created the controller, transmitter, initial datapath, and initial top level shell. Elias designed and created the FSM receiver, the buffer, and built off of Colin's datapath to make it much more robust and efficient.

Future Work: What could you add/improve (e.g., scaling, more efficient FSM, additional features)?

There are a few things we would do in the future. Firstly is to have a random hidden word pulled from ROM. We would do this by multiplying a random float from the uniform distribution by the length of our dictionary (rounding it after), and then using that value as the index in the dictionary to pull from. This would make the game more fun (and usable) than simply using the same repeated hidden word over and over. Also, Colin had the logic working but couldn't run it because he coded it after he left for Hong Kong, outputting an LED pin, one for winning the game and one for losing. This simplifies the grading process and alerts the user when the game is up. Another small thing is input validation. Converting to lowercase (if input was upper case) and then seeing if it lines up with the true hidden word - thus making the game case sensitive like the true Wordle. Alongside this is making sure all character inputs are valid letters, and not a non-numeric character. Lastly, a reset would be fun so that you could keep playing over and over again without having to reprogram the device each game.

Acknowledgements

Crediting Josh and Tad for helping us design. Crediting ChatGPT for helping with design as well, and occasionally debugging errors (helping find what was broken). Google for telling us what to connect PuTTy to in the constraints.

Conclusions

Our project was ultimately successful. We were able to design, implement, and test a full system with both serial communication interface transmitter and receiver modules, as well as integrate

them into the properly functioning Wordle project. Each component was validated through simulation or by hand in case of the algorithm, showing correct state sequencing, proper timings, and data transfer between subcomponents and the PuTTy terminal. The system followed standard UART protocol, and our test benches confirmed that both transmission and reception produced accurate results. While there were challenges in debugging timing and mostly printing the proper characters to the terminal when we wanted it, the final design worked as intended and demonstrated that our approach was sound. Also, we successfully reverse engineered the Wordle algorithm to check if a guess was correct and provide the proper feedback.

Key takeaways from this project include the importance of designing each component out well in advance of coding. We also learned that writing helpful and thorough test benches is essential, as we were originally just recompiling to see if it would work - and that took up much needed time. For future students, our advice is to start testing each module in isolation before combining them, to design extremely robustly beforehand, and to start early. Our team was hindered by Colin having to leave extremely early, and we would have been able to complete several of the future work pieces otherwise. Our advice, as is similar to what we heard, is to break the problem down into smaller, verifiable steps first, and then build up from there.

Appendix A

buffer5x8.vhd - buffer
constraints.xdc - constrain
controller.vhd - controller
datapath.vhd - datapath
sci_receiver.vhd - receiver
sci_shell.vhd - shell
sci_transmitter.vhd - transmitter

Appendix B

controller_tb.vhd. It tests the controller states and output control signals. See Section 3 controller subheading.
Sci_transmitter_tb.vhd. It tests that the transmitter sends out the proper signals based on an inputted parallel register. See Section 3 SCI Transmitter subheading.
Top_shell_tb.vhd. It tests the top level shell and all its inner workings. See Section 3, Top Shell subheading.

Appendix C

Computer Generated RTLs